

Bài tập về Lớp trừu tượng, đa hình, tổng quát

Bài 1: The template shape

Mô tả:

1. Thiết kế Abstract Class **Shape**:

- Thuộc tính `protected int x, y`.
- Constructor khởi tạo toạ độ theo x, y.
- Hai phương thức abstract: `public abstract void draw()` và `public abstract void erase()`.
- Phương thức thường: `public void moveTo(int newX, int newY)`.
 - Logic của hàm này **bắt buộc** theo thứ tự:
 1. Gọi `erase()`.
 2. Cập nhật `x, y` sang toạ độ mới.
 3. Gọi `draw()`.

2. Thiết kế Class **Con**:

- Tạo lớp `Circle` và `Square` kế thừa `Shape`.
- Hiện thực hóa `draw()` in ra *"Vẽ hình tròn tại..."* và `erase()` in ra: *"Xóa hình tròn tại..."*.

3. Thực hành:

- Trong `main`, tạo một `Circle` tại (10, 10).
- Gọi lệnh `circle.moveTo(20, 20)`.
- Quan sát output để thấy luồng chạy tự động: Xóa -> Cập nhật -> Vẽ lại.

4. Ví dụ:

- **Input:** (No input)
- **Output:**

Xóa hình tròn tại (10, 10)

Vẽ hình tròn tại (20, 20)

Bài 2: Super hero - đa kế thừa Interface

Mô tả:

1. Thiết kế Interface:

- **CanFly**: có hàm `void fly()`.
 - **CanSwim**: có hàm `void swim()`.
 - **CanFight**: có hàm `void fight()`.
2. **Thiết kế Class:**
- Lớp **ActionCharacter** (lớp thường): có hàm `void fight()` (In ra: "Đấm bốc...").
 - Lớp **Hero** kế thừa **ActionCharacter** và implements **CanFly**, **CanSwim**, **CanFight**.
 - *Lưu ý:* Vì **ActionCharacter** đã có hàm `fight()`, lớp **Hero** có cần implement lại `fight()` của interface **CanFight** không? Hãy thử nghiệm.
3. **Thực hành:**
- Viết chương trình tạo một **Hero**.
 - Thử ép kiểu **Hero** về **CanSwim** để gọi hàm `swim()`.
 - Thử ép kiểu **Hero** về **CanFight** để gọi hàm `fight()`.
4. **Ví dụ:**
- **Input:** (No input)
 - **Output:**
- Hero is swimming
- Đấm bốc...

Bài 3: Employee management

Mô tả:

1. **Thiết kế Interface **IWorkable**:**
 - Phương thức `void work()`.
2. **Thiết kế Abstract Class **Employee** implements **IWorkable**:**
 - Thuộc tính: `id`, `name`, `baseSalary`.
 - Phương thức abstract: `double calculatePay()`.
 - *Lưu ý:* Class **Employee** có bắt buộc phải implement hàm `work()` của interface không? (Thử nghiệm để trả lời).
3. **Thiết kế Class Con:**
 - **OfficeWorker** (Nhân viên văn phòng):
 - `calculatePay() = baseSalary`.
 - `work() = in ra "Soạn thảo văn bản"`.
 - **Technician** (Kỹ thuật viên):

- Có thêm thuộc tính `overtimeHours`.
- `calculatePay() = baseSalary + overtimeHours * 20000`.
- `work()` = in ra "*Lắp đặt thiết bị*".

4. Thực hành:

- Tạo danh sách nhân viên hỗn hợp.
- Duyệt danh sách, tính tổng lương và yêu cầu mọi người làm việc (`work()`)

5. Ví dụ:

- **Input:** (No input)

3

O E01 NguyenVanA 5000000

T E02 TranThiB 5000000 30

O E03 LeVanC 4500000

(Dòng đầu tiên nhập vào một số n là số nhân viên

Các dòng tiếp theo mỗi dòng nhập thông tin của một nhân viên.

Nếu là OfficeWorker:

O [id] [name] [baseSalary]

Nếu là Technician:

T [id] [name] [baseSalary] [overtimeHours])

- **Output:**

NguyenVanA - Pay: 5000000.0

Soạn thảo văn bản

TranThiB - Pay: 5600000.0

Lắp đặt thiết bị

LeVanC - Pay: 4500000.0

Soạn thảo văn bản

Total Pay = 15100000.0

Bài 4: The access modifier trap

Yêu cầu:

1. Gõ đoạn code sau vào IDE/Text Editor:

```
interface IData {  
    void show(); // Mặc định là public abstract  
}  
  
class DataManager implements IData {  
    // Cố tình KHÔNG ghi public  
    void show() {  
        System.out.println("Show Data");  
    }  
}
```

2. Biên dịch. Lỗi gì xuất hiện?
3. Sửa lỗi và giải thích tại sao.
4. Ví dụ:
 - **Input:** (No input)
 - **Output:**
 - + Trước khi sửa:

Error ...

- + Sau khi sửa:

Show Data

Bài 5: The generic pair

Mục tiêu: Làm quen với Generic Class Hiểu cách tham số hóa kiểu dữ liệu.

Mô tả:

1. Tạo class **Pair<K, V>**:
 - o Thuộc tính: **private K key, private V value.**

- Constructor: `public Pair(K key, V value)`.
 - Methods: `getKey()`, `getValue()`, `setKey(K key)`, `setValue(V value)`.
 - Override `toString()`: Trả về dạng `"key - value"`.
2. **Ứng dụng thực tế (trong hàm main):**
- Tạo `Pair<String, Integer>` để lưu thông tin: ("Tuổi", 20).
 - Tạo `Pair<String, String>` để lưu thông tin: ("Mã SV", "SV001").
 - Tạo `Pair<Integer, Double>` để lưu tọa độ địa lý: (105, 21.5).
3. **Thử nghiệm lỗi:**
- Cố tình gán sai kiểu dữ liệu (ví dụ: gán chuỗi vào Pair đang định nghĩa là Integer) để xem trình biên dịch báo lỗi như thế nào ngay lúc gõ code.
4. **Ví dụ:**
- **Input:** (No input)
 - **Output:**
- Tuổi - 20
- Mã SV - SV001
- 105 - 21.5

Bài 6: The universal sorter

Mục tiêu: Viết thuật toán một lần, chạy cho mọi kiểu dữ liệu.

Mô tả:

1. **Tạo class tiện ích `ArrayUtils`:**
 - Viết phương thức `public static <T> void swap(T[] array, int i, int j)`: Hoán đổi vị trí 2 phần tử trong mảng.
 - Viết phương thức **Sắp xếp nổi bọt (Bubble Sort)** tổng quát: `public static <T extends Comparable<T>> void sort(T[] array)`.
 - *Lưu ý:* Cú pháp `<T extends Comparable<T>>` nghĩa là kiểu T bắt buộc phải có khả năng so sánh (implements Comparable).
2. **Thử nghiệm:**
 - Tạo mảng `Integer[]`: {5, 1, 3, 2} -> Gọi `sort` -> In kết quả.
 - Tạo mảng `String[]`: {"Java", "C++", "Python"} -> Gọi `sort` -> In kết quả.
 - Tạo mảng `Student[]` -> Gọi `sort`.

3. Ví dụ:

- **Input:** (No input)
- **Output:**

1 2 3 5

C++ Java Python

Bài 7: Lambda revolution

Mô tả: Sử dụng lại danh sách `Student` (id, name, gpa) ở bài tập lần trước.

1. Filter (Lọc):

- Viết hàm `removeIf` của List kết hợp Lambda để xóa các sinh viên có GPA < 5.0.
- Code: `students.removeIf(student -> student.getGpa() < 5.0);`

2. Sort (Sắp xếp tùy biến):

- Sử dụng `Collections.sort` hoặc `List.sort` với Lambda để sắp xếp sinh viên theo tên.
- Code cũ: `new Comparator<Student>() { ... }` -> Code mới: `(s1, s2) -> s1.getName().compareTo(s2.getName())`.

3. Custom Functional Interface:

- Tạo interface `Operation<T>` có 1 hàm `T execute(T a, T b)`.
- Trong main, dùng Lambda định nghĩa phép cộng, trừ, nhân, chia số thực thông qua interface này.

4. Ví dụ:

- **Input:**

4

S01 NguyenVanA 7.5

S02 TranThiB 4.5

S03 LeVanC 8.0

S04 PhamVanD 5.0

- **Output:**

After removing GPA < 5.0:

S01 NguyenVanA 7.5

S03 LeVanC 8.0

S04 PhamVanD 5.0

After sorting by name:

S03 LeVanC 8.0

S01 NguyenVanA 7.5

S04 PhamVanD 5.0

Bài 8: Hệ thống nhà thông minh

Mục tiêu: Phân biệt rõ khi nào dùng **Abstract Class** và khi nào dùng **Interface**.

Bài toán: Một bộ điều khiển trung tâm (Hub) cần quản lý các thiết bị điện trong nhà.

1. Các loại thiết bị:

- **Đèn thông minh:** Có thể Bật/Tắt và Tăng/Giảm độ sáng.
- **Máy lạnh:** Có thể Bật/Tắt và Kết nối Wifi để điều khiển từ xa.
- **Loa thông minh:** Có thể Bật/Tắt, Tăng/Giảm âm lượng và Kết nối Wifi.
- **Rèm cửa tự động:** Chỉ có thể Mở/Đóng (coi như Bật/Tắt), không có Wifi.

2. Yêu cầu quản lý:

- Tất cả thiết bị đều có: Mã định danh, Tên, Trạng thái (On/Off).
- Hub có chức năng: "**Turn Off All**" (Tắt toàn bộ thiết bị trong nhà dù là đèn hay rèm).
- Hub có chức năng: "**Setup Wifi**" (Chỉ những thiết bị nào hỗ trợ Wifi mới được cấu hình, thiết bị khác thì bỏ qua).

Yêu cầu:

- Hãy thiết kế sơ đồ lớp để Hub quản lý được danh sách thiết bị hỗn hợp này.
- *Gợi ý:* Khả năng "Kết nối Wifi" không phải thiết bị nào cũng có. Khả năng "Điều chỉnh mức độ" (sáng/âm lượng) cũng vậy.
- Dựa theo sơ đồ lớp đã thiết kế, hay cài đặt hệ thống bằng ngôn ngữ Java.

Ví dụ:

- **Input:**

4

L 01 LivingRoomLight

AC 02 BedroomAC

S 03 SmartSpeaker

C 04 WindowCurtain

- **Output:**

Turn Off All Devices:

LivingRoomLight turned off

BedroomAC turned off

SmartSpeaker turned off

WindowCurtain turned off

Setup Wifi:

BedroomAC connected to wifi

SmartSpeaker connected to wifi

Bài 9: Kho hàng tổng quát

Mục tiêu: Ứng dụng **Generics** để viết một lớp quản lý dùng chung cho nhiều loại dữ liệu, kết hợp với **Bounded Type Parameters**.

Bài toán: Bạn cần viết phần mềm cho một hệ thống kho bãi.

1. Kho đơn năng:

- Có kho chỉ chuyên chứa **Thực phẩm** (để kiểm soát hạn sử dụng).
- Có kho chỉ chuyên chứa **Đồ điện tử** (để kiểm soát chế độ bảo hành).
- Không được phép để lẫn lộn (Ví dụ: Không thể nhét Tivi vào kho Thực phẩm).

2. Quy tắc quản lý:

- Mỗi kho đều có chức năng: **Nhập kho**, **Xuất kho**, và **Kiểm kê** (In danh sách).
- Tuy nhiên, khi **Kiểm kê**:
 - Kho Thực phẩm phải in ra: "Tên hàng - Hạn sử dụng".
 - Kho Điện tử phải in ra: "Tên hàng - Thời gian bảo hành".
- Tất cả các mặt hàng đều phải có Mã hàng và Tên hàng.

Yêu cầu:

- Ứng dụng Generics để thiết kế hệ thống hợp lý, vẽ sơ đồ lớp.
- Sử dụng **Bounded Type** (`<T extends ...>`) để đảm bảo Kho chỉ chứa những thứ là "Hàng hóa" (Product), không chứa linh tinh (như String hay Integer).
- Dựa theo sơ đồ lớp đã thiết kế, hay cài đặt hệ thống bằng ngôn ngữ Java.

Bài 10: Hệ thống Quản lý Thư viện Đa phương tiện

Mục tiêu: Sử dụng Generics với Bounded Type Parameters kết hợp với Abstract Class và tính Đa hình để quản lý các loại tài liệu khác nhau.

Bài toán: Bạn cần xây dựng một hệ thống quản lý cho một thư viện hiện đại.

- **Các loại tài liệu:** * Thư viện có chứa **Sách** (Book) và **Đĩa DVD** (DVD).
 - Tất cả tài liệu đều là **MediaItem** (có Mã tài liệu, Tên tài liệu).
- **Quy tắc quản lý:**
 - Mỗi khu vực trong thư viện (LibrarySection) chỉ chuyên lưu trữ một loại tài liệu nhất định (Ví dụ: Khu vực Sách không được để lẫn Đĩa DVD).
 - Mỗi khu vực đều có các chức năng: Thêm tài liệu, Xóa tài liệu, và Hiển thị danh sách tài liệu đang có.
 - Tuy nhiên, khi hiển thị thông tin:
 - Đối với **Sách**, hệ thống phải in ra: "Tên sách - Tác giả - Số trang".
 - Đối với **Đĩa DVD**, hệ thống phải in ra: "Tên đĩa - Đạo diễn - Thời lượng (phút)".

Yêu cầu:

1. Thiết kế sơ đồ lớp (Class Diagram) thể hiện rõ Abstract Class, các lớp con và lớp Generic quản lý khu vực.

2. Sử dụng Generics `<T extends MediaItem>` cho lớp `LibrarySection` để đảm bảo khu vực chỉ chứa các đối tượng là tài liệu hợp lệ.
3. Cài đặt hệ thống bằng ngôn ngữ Java, viết hàm `main` để tạo dữ liệu giả lập và kiểm thử các chức năng.